

PayEdu (ETUTRANSFERT) — Documentation

Backend

Destinée au développeur frontend : ce document décrit tout ce qui est implémenté côté backend à ce jour, pour que le développement du frontend reste en accord avec l'API réelle. À mettre à jour à chaque évolution du backend.

Stack : FastAPI (Python, synchrone) + PostgreSQL + SQLAlchemy + Alembic **Architecture** : monolithe modulaire (voir note en bas de document sur la décision d'architecture)

1. Vue d'ensemble

Le backend couvre actuellement deux domaines fonctionnels :

1. **Authentification** — 4 rôles (Admin, Établissement, Apprenant, Caisse), JWT, TOTP
2. **Paiement** — catalogue de frais, quitus, paiement via CinetPay (Mobile Money + Carte), validation caisse par QR codes, reçu PDF

Tout est servi par une seule application FastAPI, sous un préfixe commun (à adapter selon ta config, ex. `http://localhost:8000`).

2. Authentification

2.1 Les 4 rôles

Rôle	Valeur (<code>role</code> dans le JWT)	Se connecte avec
Admin	<code>admin</code>	email
Établissement	<code>etablissement</code>	email
Apprenant	<code>apprenant</code>	matricule ou email
Caisse	<code>caisse</code>	email

Hiérarchie de création : Admin → crée les Établissements → crée les Apprenants et Caisses de son établissement. Un apprenant/une caisse ne peut pas s'auto-inscrire ; c'est l'établissement qui crée ces comptes.

2.2 Endpoints

Méthode	Route	Rôle requis	Description
POST	/auth/register/admin	public	Crée un admin (à restreindre en production)
POST	/auth/register/etablissement	admin	Crée un établissement + son compte gestionnaire
POST	/auth/register/apprenant	etablissement	Crée un compte apprenant
POST	/auth/register/caisse	etablissement	Crée un compte caisse
POST	/auth/login	public	Connexion → renvoie le JWT
GET	/auth/totp/setup	authentifié	Renvoie le QR code TOTP à scanner (une fois)
GET	/auth/me	authentifié	Profil de l'utilisateur connecté

2.3 Connexion — POST /auth/login

```
// Requête
{
  "identifiant": "21T00123",
  "mot_de_passe": "MotDePasse123"
}

// Réponse 200
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...\"",
  "token_type": "bearer",
  "role": "apprenant"
}
```

2.4 Utiliser le token

Toutes les routes protégées attendent :

```
Authorization: Bearer <access_token>
```

Le JWT contient : `sub` (UUID utilisateur), `role`, `etablissement_id`, `exp`.

2.5 TOTP (double authentification pour le paiement)

- Un `totp_secret` est généré automatiquement à la création d'un compte **apprenant** (pas les autres rôles).
- `GET /auth/totp/setup` renvoie `{ "provisioning_uri": "...", "qrcode_base64": "...", "qr_code": "..." }` — le frontend doit afficher ce QR code **une seule fois**, à un moment approprié (fin d'inscription, ou dans les paramètres du compte), pour que l'apprenant le scanne avec une app d'authentification (Google Authenticator ou équivalent).
- Ce TOTP est utilisé **uniquement** au moment de l'autorisation finale d'un paiement (pas à la connexion normale).

3. Catalogue de frais et quitus

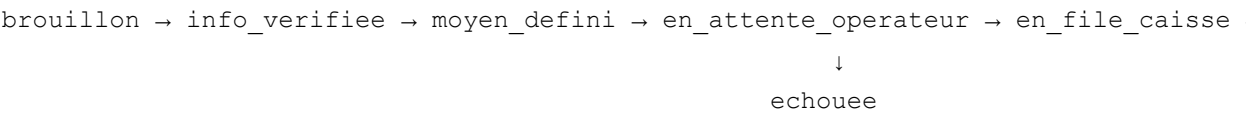
Un établissement définit ses propres frais (montant, échéance) ; il dépose aussi les quitus de ses apprenants. **L'apprenant ne uploadé jamais de fichier** — il sélectionne un objet de paiement pour lequel un quitus a déjà été déposé.

Méthode	Route	Rôle requis	Description
POST	<code>/objets-paiement</code>	etablissement	Crée un frais (libellé, montant, échéance, phase)
GET	<code>/objets-paiement</code>	apprenant	Liste les frais du propre établissement de l'apprenant
POST	<code>/objets-paiement/{id}/quitus</code>	etablissement	Dépose/remplace le quitus d'un apprenant pour ce frais
GET	<code>/quitus/moi</code>	apprenant	Liste les quitus déjà déposés pour l'apprenant connecté

Important pour le frontend : avant d’afficher le Formulaire 1 du paiement, il faut avoir chargé `GET /objets-paiement` pour peupler la liste des frais disponibles. Si l’apprenant tente de créer un paiement pour un objet sans quitus déposé, `POST /payments` renvoie un `400 explicite`.

4. Le parcours de paiement

4.1 Machine à états



(`annulee` possible depuis les premiers statuts, non encore exposé via un endpoint dédié)

4.2 Étapes et endpoints, dans l’ordre

#	Écran frontend	Méthode	Route	Rôle
1	Sélection frais + moyen de paiement	POST	<code>/payments</code>	apprenant
2	Vérification des infos (issues du quitus)	PATCH	<code>/payments/{id}/informations</code>	apprenant
3	Infos moyen de paiement	PATCH	<code>/payments/{id}/moyen-paiement</code>	apprenant
4	Récapitulatif	GET	<code>/payments/{id}/recapitulatif</code>	apprenant
5	Autorisation (code TOTP)	POST	<code>/payments/{id}/autoriser</code>	apprenant
6	Attente / statut	GET	<code>/payments/{id}</code>	apprenant
7	Téléchargement du reçu	GET	<code>/payments/{id}/recu</code>	apprenant

4.3 Détail des requêtes/réponses

Étape 1 — `POST /payments`

```
// Requête
{
  "etablissement_id": "76acdad1-...",
  "objet_paiement_id": "3f2e1a90-...",
  "moyen_paiement": "orange_money" // ou "mtn_momo" | "carte_bancaire"
}
// → 400 si aucun quitus n'existe pour ce couple (apprenant, objet_paiement)
```

Étape 2 — PATCH /payments/{id}/informations

```
{
  "infos_confirmees": {
    "nom": "Fotso Marie",
    "matricule": "21T00123",
    "filiere": "Génie logiciel",
    "niveau": "Licence",
    "date_naissance": "2002-04-15"
  }
}
```

Étape 3 — PATCH /payments/{id}/moyen-paiement

```
{
  "numero_compte_paiement": "677123456"
}
```

Si `moyen_paiement == "carte_bancaire"` (choisi à l'étape 1), **4 champs supplémentaires deviennent obligatoires** — la requête échoue en 400 sinon :

```
{
  "numero_compte_paiement": "677123456",
  "email_paiement": "marie.fotso@etu.cm",
  "adresse_paiement": "Rue de la Joie, Akwa",
  "ville_paiement": "Douala",
  "code_postal_paiement": "00237"
}
```

Étape 5 — POST /payments/{id}/autoriser

```
// Requête
{ "code_totp": "483920" }

// Réponse 200
{
```

```
"id": "ddl9ed0d-...",
"statut": "en_attente_operateur",
"payment_url": "https://checkout.cinetpay.com/payment/..."
}
```

Le frontend doit rediriger l'apprenant vers `payment_url` (page hébergée par CinetPay) — c'est là que le numéro de carte/le PIN mobile money est saisi, jamais dans PayEdu.

Étape 6 — Polling `GET /payments/{id}`

```
{
  "id": "ddl9ed0d-...",
  "statut": "en_file_caisse",
  "montant": "45000.00",
  "objet_paiement_id": "3f2e1a90-...",
  "moyen_paiement": "orange_money",
  "reference_transaction": "alb2c3d4-...",
  "numero_recu": "PAY-2026-0042",
  "motif_echec": null,
  "created_at": "...",
  "updated_at": "..."
}
```

`numero_recu` reste `null` tant que le statut n'est pas `acquittee`.

Recommandation : poller toutes les 3-5 secondes après l'étape 5, jusqu'à ce que `statut` soit `en_file_caisse` (succès partiel, en attente de la caisse), `acquittee` (terminé) ou `echouee`.

Étape 7 — `GET /payments/{id}/recu`

- Renvoie un **fichier PDF binaire** (`Content-Type: application/pdf`), pas du JSON.
- Uniquement disponible si `statut == "acquittee"` , sinon `409` .
- Le PDF contient : en-tête établissement, bloc bénéficiaire (nom, matricule, filière), bloc paiement (objectif, n° de reçu lisible `PAY-AAAA-NNNN` , moyen, date), montant, QR code de vérification, zone cachet/signature.

5. Côté caisse (validation finale)

Un compte de rôle `caisse` gère la validation physique du paiement, via 4 QR codes générés automatiquement après confirmation CinetPay.

Méthode	Route	Rôle	Description
GET	<code>/payments/caisse/queue</code>	caisse	Liste des paiements en_file_caisse de l'établissement
POST	<code>/payments/caisse/{id}/scan/{type_code}</code>	caisse	Marque un QR code comme scanné
POST	<code>/payments/caisse/{id}/finaliser</code>	caisse	Passe le paiement à acquittee (échoue si les 4 QR ne sont pas scannés)

Valeurs possibles de `type_code` : `infos_apprenant` , `compte_paiement` , `montant` , `identifiant_transaction` — il faut scanner les 4 avant de pouvoir finaliser.

6. Webhook CinetPay (pas d'appel frontend)

`POST /payments/webhooks/cinetpay` — appelé par CinetPay lui-même (ou simulé manuellement en dev), jamais par le frontend. Non documenté dans Swagger (`include_in_schema=False`).

7. Gestion des erreurs — codes HTTP à anticiper côté frontend

Code	Signification typique
400	Donnée invalide, ou action métier impossible (ex: quitus manquant, champs carte incomplets)
401	Token absent/invalide/expiré, ou code TOTP incorrect
403	Rôle incorrect pour cette route
404	Ressource introuvable (paiement, objet de paiement...)
409	Conflit d'état (ex: tentative de refaire une étape déjà passée, QR codes non tous scannés)

Le corps d'erreur suit le format standard FastAPI : `{ "detail": "message explicite en français" }`.

8. Authentification — précisions techniques pour le frontend

- Schéma : **Bearer token simple** (`HTTPBearer`), pas un vrai flux OAuth2 — un header `Authorization: Bearer <token>` suffit, pas besoin de `client_id/client_secret`.
 - CORS actuellement ouvert (`allow_origins=["*"]`) en développement — sera restreint au domaine réel du frontend en production.
 - Pas de refresh token pour l'instant : le token expire après `ACCESS_TOKEN_EXPIRE_MINUTES` (actuellement 60 min) et l'utilisateur doit se reconnecter.
-

9. Ce qui n'est PAS encore implémenté (ne pas construire d'écrans dessus pour l'instant)

- Dashboard établissement (statistiques, rapports)
- Système de rendez-vous/files d'attente (hors périmètre MVP, voir décision d'architecture)
- Notifications (email/SMS/push)
- Gestion de profil apprenant (modification des infos personnelles)
- Recherche/filtrage avancé des paiements

10. Décision d'architecture (contexte pour le jury / rapport)

Le CDC de référence prévoit une architecture microservices complète (8 services). Le MVP actuel est un **monolithe modulaire** respectant les mêmes frontières de domaine (dossiers `models/` , `schemas/` , `services/` , `api/` séparés par domaine : `user` , `payment` , `catalogue`), migrable vers des microservices sans réécriture complète. Un début de découpage en 2-3 services (`svc-auth` , `svc-paiement` , `svc-enregistrement`) a été exploré séparément mais n'est pas ce qui est actuellement en cours d'intégration frontend.

11. Variables d'environnement nécessaires

```
DATABASE_URL=postgresql://...  
SECRET_KEY=...  
JWT_ALGORITHM=HS256  
ACCESS_TOKEN_EXPIRE_MINUTES=60  
  
CINETPAY_APIKEY=  
CINETPAY_SITE_ID=  
CINETPAY_NOTIFY_URL=  
CINETPAY_RETURN_URL=  
USE MOCK CINETPAY=True    # laisser True tant qu'il n'y a pas de vrai compte march
```

Dernière mise à jour : reflète l'état du backend après l'ajout du catalogue de frais, des quittus, du reçu PDF avec numéro lisible, et de la correction du schéma d'authentification Swagger (HTTPBearer).